

СОРТИРОВКА МАССИВОВ

СБОРНИК ПОПУЛЯРНЫХ МЕТОДОВ СОРТИРОВКИ

Методов сортировки массивов множество, и каждый из них эффективнее остальных в конкретной ситуации. Ниже будут приведены самые известные методы сортировки массивов и описание их работы.

СОРТИРОВКА ПУЗЫРЬКОМ

Данный метод является одним из самых примитивных и простых для понимания. Допустим, имеется следующий массив, который нужно упорядочить по возрастанию:

64	53	49	71	35	27	9
----	----	----	----	----	----	---

Чтобы отсортировать его, необходимо сравнить каждую пару соседних элементов и, если они нарушают порядок (первый элемент в паре больше второго), то произвести обмен ячеек значениями. Выполним такой алгоритм, начиная с первых элементов:

64	53	49	71	35	27	9
----	----	----	----	----	----	---

Первая пара: $64 > 53$, следовательно, эти два числа меняем местами и получаем

53	64	49	71	35	27	9
----	----	----	----	----	----	---

Следующая пара: $64 > 49$, эти два числа тоже меняем местами и получаем

53	49	64	71	35	27	9
----	----	----	----	----	----	---

Следующая пара: $64 < 71$. Эти два числа менять местами не будем.

53	49	64	71	35	27	9
----	----	----	----	----	----	---

Проверяем следующую пару: $71 > 35$. Вот эти два числа точно меняем местами и получаем

53	49	64	35	71	27	9
----	----	----	----	----	----	---

Следующая пара: $71 > 27$. Эти два числа меняем местами:

53	49	64	35	27	71	9
----	----	----	----	----	----	---

И последняя пара: $71 > 9$. Эти два числа тоже меняем и получаем

53	49	64	35	27	9	71
----	----	----	----	----	---	----

По всем парам чисел мы прошли, но массив ещё явно не отсортирован. Но прогресс уже есть: самое большое число (71) уже стоит на своём законном месте.

Осталось проделать вышеописанный алгоритм столько раз, сколько всего элементов в массиве. При этом с числами, которые уже закрепились на своих местах, можно уже не составлять пары – они всё равно со своих мест не сдвинутся. Данный процесс будет выглядеть примерно так:

В конце прохода №1 у нас получилось:

53	49	64	35	27	9	71
----	----	----	----	----	---	----

После прохода №2 мы получим:

49	53	35	27	9	64	71
----	----	----	----	---	----	----

После прохода №3:

49	35	27	9	53	64	71
----	----	----	---	----	----	----

После прохода №4:

35	27	9	49	53	64	71
----	----	---	----	----	----	----

После прохода №5:

27	9	35	49	53	64	71
----	---	----	----	----	----	----

И после прохода №6...

9	27	35	49	53	64	71
---	----	----	----	----	----	----

Как видно, после каждого прохода одно число (самое крупное из неотсортированных) занимает своё законное место. И, чтобы свои законные места заняли все элементы в массиве, требуется число проходов, на 1 меньшее количества элементов в массиве.

Реализация такого алгоритма на языке программирования требует использования вложенных циклов **for** и для массива по имени **arr** выглядит примерно так:

Pascal

```
for k:=1 to length(arr)-1 do
  for j:=1 to length(arr)-k do
    if (arr[j]>arr[j+1]) then begin
      tmp:=arr[j];
      arr[j]:=arr[j+1];
      arr[j+1]:=tmp;
    end;
```

C++

```
for (int k = 0; k < arr.size()-1; k++) {
  for (int j = 0; j < arr.size()-k-1; j++) {
    if (arr[j]>arr[j+1]) {
      tmp = arr[j];
      arr[j] = arr[j+1];
      arr[j+1] = tmp;
    }
  }
}
```

Python

```
for k in range(len(arr)-1):
    for j in range(len(arr)-k-1):
        if (arr[j]>arr[j+1]):
            tmp = arr[j]
            arr[j] = arr[j+1]
            arr[j+1] = tmp
```

СОРТИРОВКА ПУЗЫРЬКОМ С ФЛАГОВОЙ ПЕРЕМЕННОЙ

Данный метод практически полностью копирует предыдущий, но несколько лучше обрабатывает случаи, когда все элементы массива уже отсортированы, а циклы **for** ещё не закончили работать. В проходе добавляется флаговая переменная, которая изменит своё значение, если обмен значениями произошёл хотя бы один раз за этот проход. Если за весь проход обмена значениями не было, значит массив полностью отсортирован и выполнять следующие проходы бессмысленно, что и предотвращает флаговая переменная:

Pascal

```
for k:=1 to length(arr)-1 do
begin
    flag:=True;
    for j:=1 to length(arr)-k do
    begin
        if (arr[j]>arr[j+1]) then begin
            tmp:=arr[j];
            arr[j]:=arr[j+1];
            arr[j+1]:=tmp;
            flag:=False;
        end;
    end;
    if (flag=True) then break;
end;
```

C++

```
for (int k = 0; k < arr.size()-1; k++) {
    flag = true;
    for (int j = 0; j < arr.size()-k-1; j++) {
        if (arr[j]>arr[j+1]) {
            tmp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = tmp;
            flag = false;
        }
    }
    if (flag == true) {break;}
}
```

Python

```
for k in range(len(arr)-1):
    flag = True
    for j in range(len(arr)-k-1):
        if (arr[j]>arr[j+1]):
            tmp = arr[j]
            arr[j] = arr[j+1]
            arr[j+1] = tmp
            flag = False
    if (flag == True): break
```

СОРТИРОВКА ВСТАВКАМИ

Этот метод сортировки довольно изящный и минималистичный в плане количества операций присвоения. При сортировке вставками запоминается один сортируемый элемент и размещается на своё место в ранее упорядоченной последовательности.

Для демонстрации работы данного метода возьмём ранее использованный массив:

64	35	49	71	53	27	9
----	----	----	----	----	----	---

Для начала сортируется минимально возможная последовательность – два элемента:

35	64	49	71	53	27	9
----	----	----	----	----	----	---

Затем в эту последовательность включается третий элемент и также занимает в ней своё место:

35	49	64	71	53	27	9
----	----	----	----	----	----	---

То же самое ждёт и четвёртый элемент, хотя и в данном случае перестановок нет:

35	49	64	71	53	27	9
----	----	----	----	----	----	---

Затем в последовательность добавляется пятый элемент:

35	49	53	64	71	27	9
----	----	----	----	----	----	---

Затем шестой:

27	35	49	53	64	71	9
----	----	----	----	----	----	---

И последний:

9	27	35	49	53	64	71
---	----	----	----	----	----	----

Для реализации такого алгоритма также требуются вложенные циклы, но работают они уже несколько иначе:

Pascal

```
for k:=2 to length(arr) do
begin
  tmp:=arr[k];
  j:=k-1;
  while (j>=1) and (tmp<arr[j]) do
  begin
    arr[j+1]:=arr[j];
    j:=j-1;
  end;
  arr[j+1]:=tmp;
end;
```

C++

```
for (int k = 1; k < arr.size(); k++) {
  tmp = arr[k];
  j = k-1;
  while ((j>=0) && (tmp<arr[j])) {
    arr[j+1] = arr[j];
    j = j-1;
  }
  arr[j+1] = tmp;
}
```

Python

```
for k in range(1, len(arr)):
  tmp = arr[k]
  j = k-1
  while (j>=0) and (tmp<arr[j]):
    arr[j+1] = arr[j]
    j = j-1
  arr[j+1] = tmp
```

ДРУГИЕ МЕТОДЫ СОРТИРОВКИ

Методов сортировки массивов очень много. Каждый из них может быть эффективнее других в одних случаях и проигрывать в других. Нет ни одного метода сортировки, который будет самым-самым абсолютно всегда – иначе мы бы только его и показали. В противовес этому, существуют методы сортировки, эффективность которых крайне низкая и практически всегда проигрывает другим методам. Такие методы обычно разрабатываются программистами исключительно ради шутки, но порой даже такие вещи удостаиваются собственной статьи на Википедии.